



Penetration Testing

Moonberg (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	39
Our Methodology	40
Disclaimers	42
About Hashlock	43

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Moonberg team partnered with Hashlock to conduct a security Penetration Test of their Moonberg Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Moonberg is an AI-powered Web3 analytics and trading platform that serves as an intelligence layer for crypto markets. It aggregates on-chain data, market metrics, wallet activity, and AI-generated trading signals across multiple blockchains into a unified terminal, enabling users to research digital assets, monitor market trends, automate trading strategies, and execute non-custodial trades from a single interface. The platform is designed to improve decision-making through advanced analytics, proprietary metrics, and AI-driven automation for both retail and professional crypto traders.

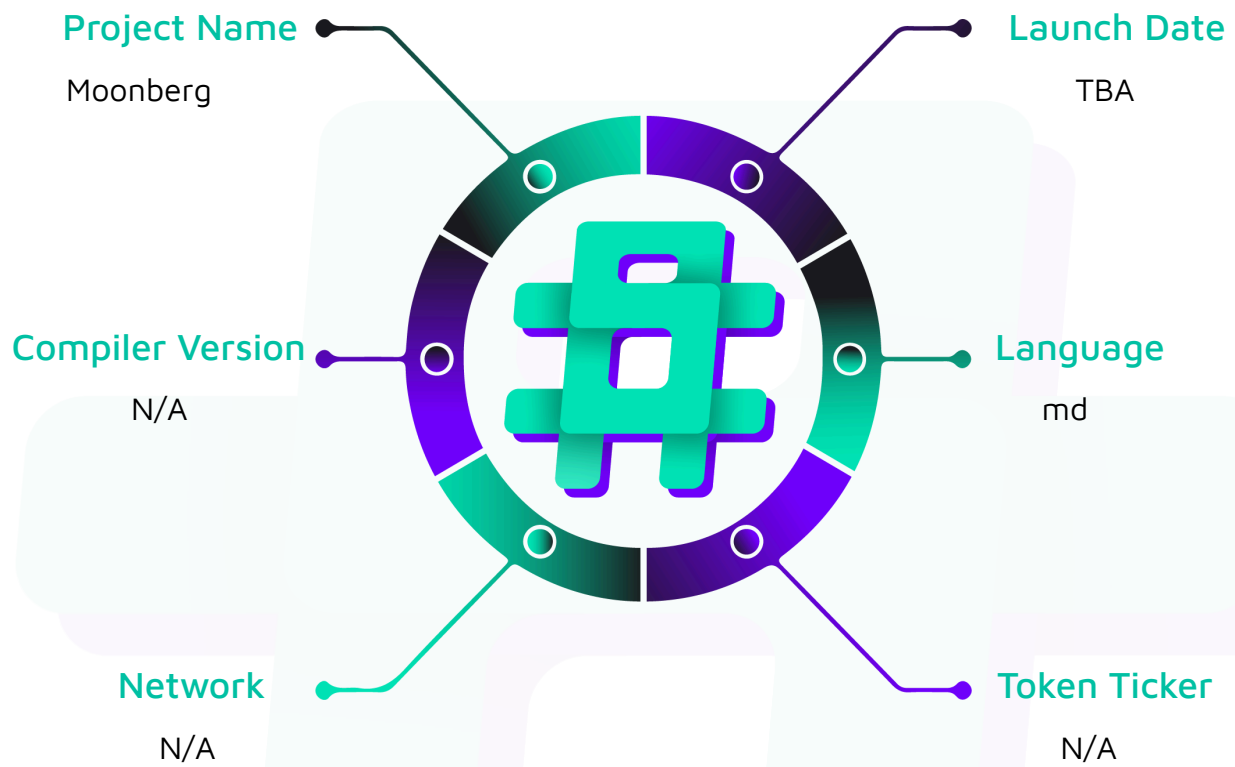
Project Name: Moonberg

Project Type: Defi

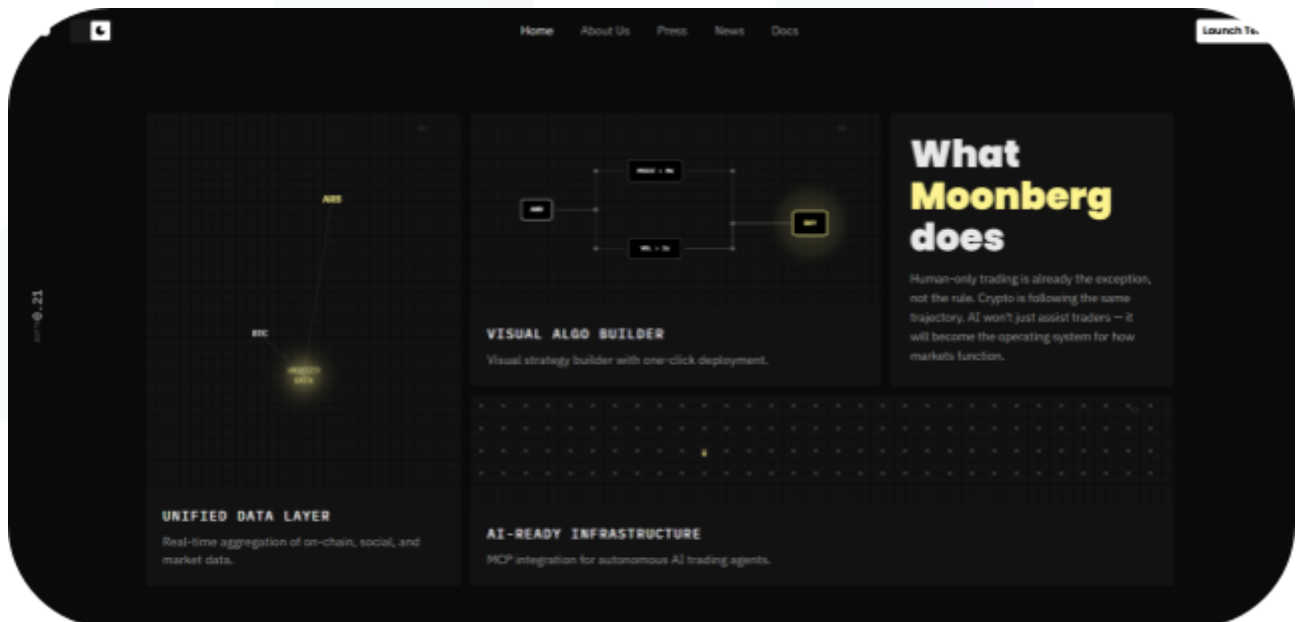
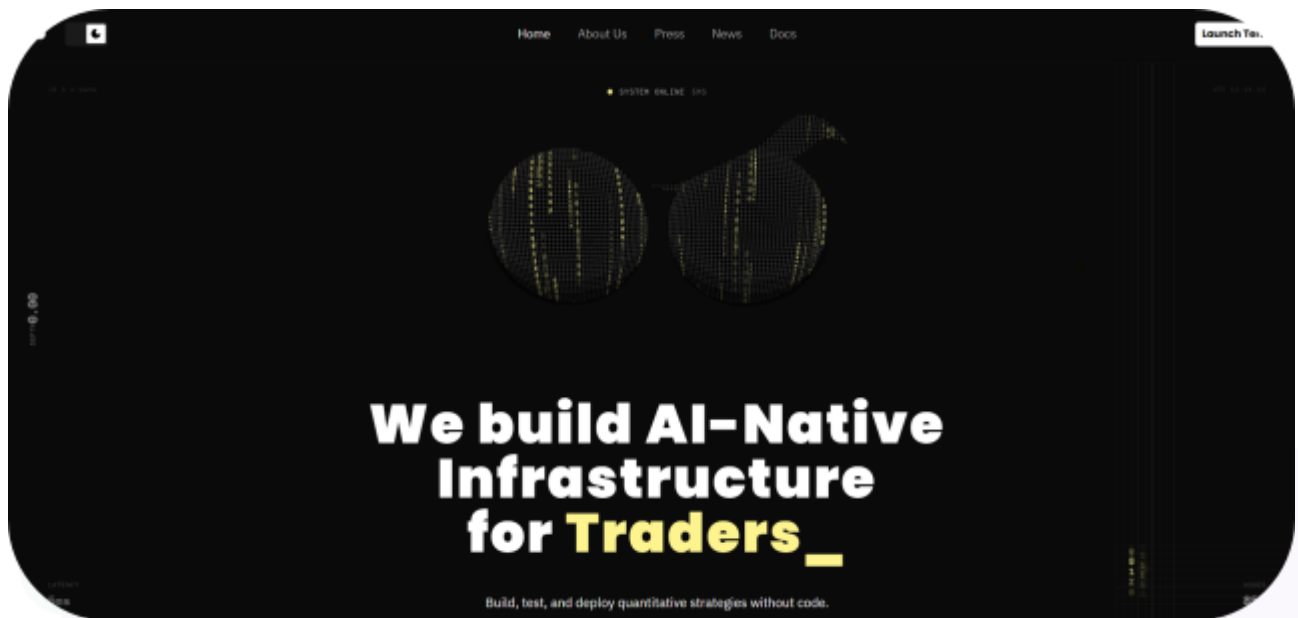
Website: <https://moonberg.com/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Moonberg project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Moonberg Project code base
Platform	Md
Penetration Test Date	July, 2026
Repository/Website URL/IP Tested	https://github.com/moonberg-ai/presale
GitHub Commit Hash (If applicable)	2f10494331f45c28304bbb662095ae9406c69400
Fix Review GitHub Commit Hash (If applicable)	bf4384944fb6fdce85273bb867a3091887d782d7

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 Medium-severity vulnerabilities

6 Low-severity vulnerabilities

2 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

[M-01][presale] allocation-engine#REFUND_REVIEW - manual refund and reconciliation flows record no completion state, allowing double or missed refunds

Description

Three deposit classes are resolved by an out-of-band cold-wallet transfer performed by a human: REFUND_REVIEW sold-out leftovers, orphan deposits to an already-finalized order, and unattributed deposits whose sender matches no PENDING order. For all three the refundable figure is only ever derived, and no database state records whether the refund has actually been paid.

Documentation Basis

"There is intentionally no dedicated database row for the leftover - it is always derivable ... so the refund amount cannot drift out of sync with the allocation history; the warning is the alerting trail, the derivation is the source of truth."

05-allocation-engine.md

Not in line: 09-operations-and-administration.md claims the order row documents what was credited and what was returned, yet no field records that the refund was paid and 02-data-model.md fixes REFUND_REVIEW as terminal (No code path moves an order out of ALLOCATED or REFUND_REVIEW), so the payout is entirely off-ledger with none of the exactly-once protection the credit path enjoys.

Vulnerability Details

The refundable amount is computed as `usdcReceived - sum(usdcSpent)` over the order's allocations. The order remains in `REFUND_REVIEW` with the same derivable figure whether or not the operator has already refunded it. There is no `refundTxHash` stored against the order, no terminal `REFUNDED` state, and nothing that links the on-chain refund back to the order. orphan and unattributed deposits have no database row at all beyond a log line and a counter.

```
// 05-allocation-engine.md - refundable is derived, never marked settled:
refundable = usdcReceived - sum(usdcSpent) over the order's PresaleAllocation
rows
// status set to REFUND_REVIEW; no REFUNDED terminal state, no refundTxHash
column.
// The same figure remains "owed" after the refund is paid out.
```

Impact

A refund can be paid twice because nothing marks it complete, directly losing treasury funds, and a refund can equally be forgotten because there is no worklist that closes out, stranding a buyer's money in the cold wallet. The absence of a refund audit trail also means the system cannot prove from its own records that an obligation was discharged.

Recommendation

Add a durable refund record: a `refundTxHash` and a settled status (a `REFUNDED` terminal state or a dedicated refund table) written when the operator records the on-chain refund, with a uniqueness constraint on the refund transaction so one obligation cannot be settled twice. Drive refunds from a queue that closes items out, so an unrefunded `REFUND_REVIEW`, orphan, or `unattributed` deposit stays visible until it is provably paid.

Status

Resolved

[M-02][presale] presalePaused#pause-check - the pause kill-switch cannot stop crediting and fails open on a flag-read error

Description

The `presalePaused` kill-switch blocks new order creation only. The deposit observers consult the flag nowhere and keep crediting any USDC that reaches the cold wallet, and the cold-wallet address is disclosed to every buyer, so a buyer who already holds the deposit address can keep depositing while the sale is paused and be allocated at the current phase parameters.

Documentation Basis

"Fails open. If the flag read errors, the check logs a warning and treats the sale as unpaused ... a Redis/DB outage in the flag path cannot accidentally halt the sale, but equally cannot be relied upon to enforce a pause during such an outage - the authoritative stop for a serious incident is the admin action plus the allocation engine's own cap enforcement" 03-api.md

Not in line: The nominated fallback, cap enforcement, prevents exceeding a phase `capUsdc` but does nothing about selling at a wrong price within it, and 09-operations-and-administration.md confirms the observers consult the pause flag nowhere, so the one incident class that warrants the kill-switch, a mispricing, cannot actually be halted.

Vulnerability Details

The pause check runs inside the order-creation route and rejects with `PRECONDITION_FAILED` while the flag is on. The observers and the allocation engine never read the flag. The check reads only the master `enabled` field and, on any flag-store read failure, logs a warning and treats the sale as unpaused. Propagation across API instances is bounded by a 5-second in-memory cache, so other instances can still accept new orders for up to that window after a pause lands.

```
// 03-api.md / 09-operations-and-administration.md
if (presalePaused.enabled) throw new TRPCErrror({ code: 'PRECONDITION_FAILED' });
// createOrder only
// observers never read the flag -> crediting continues while paused
// flag read failure -> log warning and treat sale as UNPAUSED (fail-open)
```

Impact

There is no mechanism that halts allocation at the current phase parameters during an incident. If a phase is mispriced or an attack is in progress, the operator cannot stop deposits from being credited, only stop the widget from issuing new orders, and a flag-store outage silently disables even that partial protection. The kill-switch cannot be relied on as an emergency stop for the money path.

Recommendation

Add an emergency-freeze control that the allocation engine itself honours, routing deposits that arrive while frozen into a review state rather than allocating them, so allocation at possibly-wrong parameters can be halted without stranding buyer funds. Make the kill-switch fail closed for the money-critical path, or require explicit operator acknowledgement when the flag store is unavailable.

Status

Resolved

[M-03][presale] auth#application-token - the symmetric HS256 session secret is a single point of full user and admin impersonation

Description

The application token is an HS256 JWT signed with the symmetric `JWT_SECRET` and verified locally on every request, with identity taken from the `sub` claim. An attacker who obtains `JWT_SECRET` can mint a token for any `userId`, and because the admin gate looks up the `Admin` row by the `sub` the token asserts, minting a token with a known admin's `userId` yields that admin's live `presale.manage` grant.

Documentation Basis

"No self-asserted identity. The only client-supplied identity input is the Privy identity token, verified against Privy's published JWKS; the linked-accounts set that drives wallet ownership is re-fetched server-to-server from Privy, never taken from client-controlled data." 08-authentication.md

Not in line: That property holds only for the Privy identity token. The application token's `sub` is trusted on every request and the admin lookup is keyed on it, so a leaked `JWT_SECRET` makes `sub`, including an admin's, fully self-asserted, a threat the docs never model and (per `claims` are as fresh as the token) cannot revoke within the 3-day lifetime.

Vulnerability Details

Steady-state requests verify the token against `JWT_SECRET` with the algorithm list pinned to `['HS256']`. The live admin check re-reads the `Admin` row from the database, but it trusts the token's `sub` to identify the caller, so the live check confirms the forged identity's real permissions rather than rejecting the forgery. There is no revocation or rotation path for an already-issued application token within its default 3-day lifetime.

```
// 08-authentication.md - application token
// HS256 JWT signed with JWT_SECRET; verify list pinned to ['HS256']; sub = user
id
// admin gate: fetch Admin row live from DB, keyed on the token's sub
// => forged token with sub = <admin did> passes the live presale.manage check
```

Impact

Compromise of the one symmetric secret escalates directly to impersonation of any buyer (reading their order data and creating orders in their name) and to full presale-admin capability over phase prices and caps. Because the secret both signs and verifies, it is a single high-value target, and a leaked buyer token cannot be revoked before it expires.

Recommendation

Move application-token signing to an asymmetric algorithm (RS256 or ES256) so the key held by API instances verifies but cannot mint tokens, shrinking the blast radius of a leak. Add a rotation strategy for the signing secret and a revocation path (a token version or session identifier checked per request), and consider a shorter lifetime plus a step-up check for admin actions.

Status

Acknowledged

[M-04][presale] deposit-observers#unattributed - payments from exchanges or unlinked wallets rely on non-cryptographic manual attribution

Description

Attribution requires the on-chain sender to equal a `buyerWallet` proven at Privy link time, but presale buyers commonly pay from a centralized-exchange withdrawal or a wallet they never linked. Every such deposit is `unattributed` and is resolved by an operator judgement rather than a cryptographic proof.

Documentation Basis

"A buyer can only ever be credited for money that left a wallet they demonstrably control" 07-security-and-trust-model.md

Not in line: 09-operations-and-administration.md calls paying from an unlinked wallet such as an exchange withdrawal the `typical` cause of an `unattributed` deposit and credits it to a buyer on operator inference. An exchange hot wallet is provably not a wallet the buyer controls, so the reconciliation path is an unacknowledged exception to the headline guarantee, applied to a case the docs themselves call typical.

Vulnerability Details

The documented resolution for an `unattributed` deposit is for an operator to check whether the paying address plausibly belongs to the same user as an open order and then credit it. An exchange hot wallet is not a linked `UserWallet` and cannot be, so the match rests on unverifiable off-chain signals. The design presents this as an edge case, but for a public presale paying from exchanges is a high-frequency pattern.

```
// 07 / 09 - typical cause of 'unattributed' is a buyer paying from a different  
wallet  
// (exchange withdrawal, or a second wallet).  
// resolution: operator matches deposit to a user's open order by inspection,  
// then credits or refunds. No signature ties the hot wallet to the buyer.
```



Impact

A large fraction of real deposits can fall outside the cryptographic attribution model and be credited on operator judgement, creating sustained manual load and an attribution-fraud surface. Two users could each claim the same exchange-sourced deposit with no cryptographic way to adjudicate, and mis-attribution credits the wrong user's entitlement against real money.

Recommendation

Give buyers a first-class way to pay from an arbitrary source with a verifiable reference (a per-order memo, or a required unique-amount nonce) that binds the deposit to the order without trusting the sender address, or explicitly disallow exchange payments in the UI with clear guidance. Define a controlled attribution workflow with a fixed evidence standard so manual crediting is auditable and cannot be double-claimed.

Status

Acknowledged

Low

[L-01][presale] presaleAdmin#createPhase/updatePhase - Money-critical phase price and cap are set by a single admin with no value bounds or dual control

Description

Phase priceUsd and capUsdc are the parameters that decide how many token units each deposit buys, and they are written through `presaleAdmin.createPhase` and `presaleAdmin.updatePhase` behind the single `presale.manage` permission. The only content validation is that `priceUsd` is a decimal string strictly greater than zero and `capUsdc` is a digit string, so one authorised actor can commit any internally consistent but wrong value.

Documentation Basis

"Even this maximally-privileged surface cannot break money invariants: ... the update-phase admin endpoint refuses counter values below the allocation-backed sum, and no admin procedure can create, modify, or delete allocation rows or credited orders." 07-security-and-trust-model.md

Not in line: That guarantee is scoped to oversell and counter integrity. The only content check on `priceUsd` and `capUsdc` is the `> 0` refinement in 03-api.md, so a wrong-but-consistent price passes every documented guard, and 08-authentication.md concedes the `danger: true` flag only makes the admin UI surface a warning rather than constraining the value.

Vulnerability Details

The create-phase input validates `priceUsd` against `^\d+(\.\d{1,18})?$` with a `> 0` refinement and `capUsdc` against `^\d+$. There is no upper or lower sanity bound, no rate-of-change check against the adjacent phase, and no second-approver requirement. The compare-and-swap on the loaded row and the allocation-backed floor defend only against concurrent writes and against reopening already-sold capacity. A single admin who sets 0.021 where 0.21 was intended, or an inflated capUsdc, passes every guard, and the allocation engine prices every subsequent deposit against that value.`

```
// create-phase input validation (03-api.md,  
09-operations-and-administration.md)  
const atomicAmount = z.string().regex(/^\d+$/);  
const priceUsd = z  
  .string()  
  .regex(/^\d+(\.\d{1,18})?$/)  
  .refine(value => Number(value) > 0, 'priceUsd must be > 0');  
// input: { phase: z.number().int().min(1), capUsdc: atomicAmount, priceUsd }  
// Only content check is "> 0": no upper bound, no delta-vs-prior-phase, no  
second approver.
```

Impact

A single erroneous or malicious phase edit mis-issues token entitlements against real, irreversible on-chain deposits. A price set too low sells the treasury allocation far below intended value to whoever deposits during the window, and a price set too high overcharges buyers. Because deposits credit continuously and the cold-wallet address is public, mispriced allocations accrue against live money before the error is likely noticed, and the resulting entitlements cannot be clawed back. This is the largest realistic financial exposure in a system where funds themselves cannot be stolen from the backend.

Recommendation

Add value-sanity bounds to the create-phase and update-phase inputs: a plausible price range and a maximum percentage change relative to the adjacent phase, rejecting out-of-band values unless explicitly overridden. Require dual control (a second `presale.manage` approver, or a time-delayed commit) for any change to `priceUsd` or `capUsdc` on a phase that can currently sell, and surface the human-readable price and cap for confirmation in the admin UI so `0.021` versus `0.21` is visible before commit.

Status

Resolved

[L-02][presale] deposit-observers#sender-attribution - sender attribution breaks for smart-contract and fee-sponsored payments

Description

Attribution assumes the on-chain sender is the buyer's externally-owned account. On EVM the observer attributes by the Transfer log's `from`, falling back to `tx.from` when the log sender is a router, and on Solana it attributes by `accountKeys[0]`, the fee payer. For a smart-contract or ERC-4337 account-abstraction payment `tx.from` is the bundler or relayer, and for a fee-sponsored Solana transaction `accountKeys[0]` is the sponsor, so neither matches a PENDING order.

Documentation Basis

"accountKeys[0] is the transaction's fee payer, i.e. the wallet that signed and paid for the transaction - the buyer, regardless of which intermediate accounts the USDC moved through ... [EVM] the fallback fetches the transaction and attributes by tx.from - the externally-owned account that signed the swap, i.e. the buyer."
06-deposit-observers.md

Not in line: The signer-equals-buyer relationship is stated unconditionally, but it holds only for EOA-signed transactions. 08-authentication.md lists External wallet as a supported login, and for an ERC-4337 or fee-sponsored payment `tx.from` and `accountKeys[0]` are the bundler or sponsor, not the buyer.

Vulnerability Details

The EVM crediting routine selects the newest `PENDING` order with `buyerWallet == lowercase(topics[1])`, then falls back to `buyerWallet == lowercase(tx.from)`. Under ERC-4337, `tx.from` is the bundler that submitted the `UserOperation`, not the buyer. On Solana the fee payer is read as `accountKeys[0]`, which a relay or fee-payer service sets to itself. Privy supports external wallets, some of which are smart-contract wallets, so this is a reachable configuration for real users.

```
// 06-deposit-observers.md
// EVM: order = newest PENDING with buyerWallet == lowercase(topics[1])
// fallback: buyerWallet == lowercase(tx.from) // ERC-4337: tx.from is the
bundler
// Solana: feePayer = accountKeys[0] // sponsored tx: this is the sponsor
```

Impact

A legitimate payment from a smart-contract wallet or via a fee sponsor is recorded as `presale.deposit.unattributed` and is not credited automatically. The buyer's funds sit in the cold wallet until an operator reconciles them, and the reconciliation cannot rely on the on-chain signer to identify the buyer.

Recommendation

Detect contract-wallet and sponsored payments explicitly (for example when both the log sender and `tx.from` fail to match, or when `tx.from` is a known bundler) and route them into a dedicated reconciliation path using any available on-chain hints such as inner transfers or the `UserOperation` sender. Document the limitation and, where possible, restrict the presale UI to payment flows whose signer is the linked wallet.

Status

Acknowledged

[L-03][presale] solana-observer#getSignaturesForAddress - cheap signature spam to the public cold-wallet ATA can freeze Solana crediting for all buyers

Description

The Solana observer drains new signatures on the cold wallet's USDC associated token account, paging at 1,000 signatures per page up to a hard bound of 20 pages (about 20,000 signatures) per 5-second tick. When that bound is hit while a cursor exists the observer processes nothing and holds the cursor until an operator intervenes, and the ATA is a public address any party can cause signatures to reference cheaply.

Documentation Basis

"Signature burst beyond the 20-page drain bound (Solana, cursor present) ... The tick processes nothing, logs a loud error, and leaves the cursor untouched every tick until an operator intervenes. ... Freezing is safe: deposits sit finalized on-chain and in the cold wallet, waiting to be scanned." 06-deposit-observers.md

Not in line: The docs treat this purely as a safety property against an organic backlog, but the watched account is a public ATA whose signature count any party can inflate cheaply, so the freeze is an attacker-inducible liveness denial-of-service that 07-security-and-trust-model.md's threat model does not consider.

Vulnerability Details

Pages walk newest to oldest, so a capped fetch is missing its oldest chunk, and processing it would advance the cursor past unfetched deposits. The observer therefore logs `signature gap exceeds one tick drain bound`, leaves the cursor untouched, and retries next tick, which stays frozen while the backlog exceeds the bound. An attacker who sustains more than about 20,000 referencing signatures between ticks holds the observer frozen, and a lower rate still delays draining.

```
// 06-deposit-observers.md - cursored cap-hit
// pages walk newest -> oldest; 20 full pages without reaching the cursor:
// process NOTHING, log 'signature gap exceeds one tick drain bound',
// leave cursor untouched, retry next tick (frozen until operator acts)
```

Impact

Deposit crediting for every Solana buyer halts until manual recovery, so paid buyers do not see their orders leave PENDING and the live sold and remaining counters stall. No funds are lost, but the crediting pipeline is a denial-of-service target driven by cheap on-chain spam against a public address.

Recommendation

Make the drain bound spam-resilient: pre-filter to signatures that actually change the cold-wallet USDC balance, raise or dynamically size the per-tick bound, and provide an automated recovery path that advances safely past confirmed-empty spam rather than requiring manual intervention. Alert on the frozen state so recovery time is bounded.

Status

Acknowledged

[L-04][presale] allocation-calculation#floor - token floor rounding depends on unverified decimal division precision

Description

Token amounts are computed as `floor(take / priceUsd)` using arbitrary-precision `Decimal` values, and the never-over-credit invariant relies on the division being exact to enough significant digits before the floor. Decimal libraries round the quotient to a configured significant-digit precision, commonly a default of 20, and the documentation does not state the configured value.

Documentation Basis

"All arithmetic in the engine uses arbitrary-precision Decimal (re-exported from the Prisma runtime); no floating point is involved anywhere on this path. ... tokens = floor(take / phase.priceUsd) # round DOWN - never over-credit"
 05-allocation-engine.md

Not in line: Decimal division is governed by a configured significant-digit precision the docs never state, and the never-over-credit invariant depends on that precision exceeding the digits a `Decimal(78,0)` amount divided by a `Decimal(38,18)` price can require.

Vulnerability Details

The columns are `Decimal(78, 0)` for amounts and `Decimal(38, 18)` for `priceUsd`. A large amount divided by a small price can require more significant digits in the quotient than a default precision retains. If a quotient sits just below an integer boundary and the precision rounds it up before `toFixedPlaces(0, ROUND_DOWN)`, the floor yields one atomic token unit more than the deposit strictly buys.

Impact

At pathological amount and price combinations the floor could over-credit by one atomic token unit, a one-unit violation of the stated no-over-credit invariant. The effect is tiny per deposit and unreachable at realistic USDC magnitudes, but it contradicts a load-bearing correctness claim if the precision is not configured high enough.

Recommendation

Confirm the decimal library's precision is set above the maximum significant digits reachable on the money path (`Decimal.set({ precision })` or equivalent) and add a property test at uint256-scale amounts against small prices asserting `tokensBought * priceUsd <= usdcSpent` exactly.

Status

Resolved

[L-05][presale] presale#createOrder - no rate limiting on order creation exposes metered aggregator quota and unbounded order growth

Description

The authenticated order-creation endpoint calls the metered 1inch and Jupiter APIs to build swap transactions and inserts a `PENDING` order row, with no documented rate limit or per-user quota and no expiry or cleanup path for `PENDING` orders.

Documentation Basis

"Unpaid orders stay PENDING indefinitely. There is no expiry, no cleanup job, and no delete path for orders in application code. A PENDING row is a cheap, harmless intent that doubles as a durable record of buy attempts" 02-data-model.md

Not in line: The docs describe unbounded `PENDING` accumulation as harmless but never mention any rate limiting or abuse control on `createOrder`, while 04-payments-and-networks.md confirms each swap-path call spends shared, metered Jupiter or 1inch quota.

Vulnerability Details

An authenticated user can call `createOrder` repeatedly. Each swap-path call consumes 1inch or Jupiter quota (shared across all users) and writes a `PENDING` row that is never reaped, so the aggregator quota and the order table both grow unbounded under abuse.

Impact

Repeated calls can exhaust the shared aggregator quota, degrading or breaking swap-order creation for all users, and accumulate unbounded `PENDING` rows. No funds are at risk, but the availability of the buy path and database hygiene are.

Recommendation

Apply a per-user and per-IP rate limit to the order-creation endpoint, monitor aggregator-quota consumption with backpressure, and add a retention or archival policy for stale `PENDING` orders.

Status

Resolved

[L-06][presale] auth#websocket-token - the application token is accepted as a WebSocket URL query parameter

Description

WebSocket connections cannot always set headers, so the server accepts the application token from a token query parameter on the connection URL. Tokens placed in URLs are routinely captured in server and proxy access logs, browser history, and referrer headers.

Documentation Basis

"WebSocket connections cannot always set headers, so the server accepts the application token from the Authorization header when present, falling back to a token query parameter on the connection URL." 08-authentication.md

Not in line: The fallback is presented only as a transport convenience; the docs never note that bearer credentials placed in a URL are routinely captured by proxy and server access logs, browser history, and referrer headers.

Vulnerability Details

The same application-token verification runs at connection time whether the token arrives in the Authorization header or the token query parameter. The query-parameter fallback means a long-lived bearer credential can be logged by any infrastructure that records request URLs.

Impact

An application token can leak through infrastructure that logs request URLs, exposing a bearer credential valid for up to 3 days. The presale's own public state subscription needs no token, but any authenticated subscription over this transport is affected.

Recommendation

Authenticate WebSocket connections with a short-lived single-use ticket exchanged for the session, or use a subprotocol header, so the long-lived application token never appears in a URL. If the fallback is retained, ensure access logging redacts the token query parameter.

Status

Resolved

QA

[Q-01][presale] deposit-observers#rpc-trust - crediting trusts a single RPC provider with no automated balance reconciliation

Description

Each chain's observer reads finalized state from a single RPC endpoint (SOLANA_RPC_URL for Solana, endpoints derived from one ALCHEMY_API_KEY for the EVM chains) and writes credit purely from what that provider reports as finalized. A compromised or faulty RPC that reports a fabricated finalized deposit would mint credit for money that never arrived.

Documentation Basis

"RPC providers / Read-only data source / ... Integrity of reads is cross-checked by the finality requirement (finalized commitment / finalized block tag) and, operationally, by reconciling recorded totals against the cold wallet's actual on-chain balance"
07-security-and-trust-model.md

Not in line: This reconciliation against the on-chain balance is named as the integrity control for a faulty RPC, but 09-operations-and-administration.md's monitoring section specifies only the presale.deposit.* counters and structured logs. No automated balance reconciliation is defined anywhere, so the control the trust model relies on is asserted but not specified.

Vulnerability Details

The trust model classifies RPC as a read-only data source whose integrity is cross-checked by the finality requirement and, operationally, by reconciling recorded totals against the cold wallet's on-chain balance. No automated reconciliation of

usdcReceived totals against the on-chain balance is specified, so the only stated cross-check is manual.

```
// 04 / 07 - finalized reads from one provider per chain; the only stated  
cross-check  
// is manual: cold-wallet balance must equal sum(usdcReceived) + flagged funds.  
// no automated periodic reconciliation job is described.
```

Impact

A single lying or buggy RPC can create token entitlements with no backing deposit, over-issuing against the presale supply until a human notices a divergence between recorded totals and the cold-wallet balance. Without an automated control the divergence can persist and grow.

Recommendation

Add an automated periodic reconciliation that compares the sum of usdcReceived (plus flagged unattributed and leftover funds) against the cold wallet's on-chain balance and alerts on divergence. For higher-value flows, cross-check finality against a second independent RPC before crediting so a single compromised provider cannot mint credit unilaterally.

Status

Resolved

[Q-02][presale] solana-swap-builder#slippageBps - explicit zero slippage is honoured on EVM but silently replaced on Solana

Description

The Solana swap builder computes `slippageBps` as `slippage ? Math.round(slippage * 100) : undefined`. Because the check is `falsey` rather than a defined check, an explicit slippage of `0` is treated as not supplied and Jupiter's server-side default applies, whereas the EVM path forwards an explicit `0` to 1inch.

Documentation Basis

*"because the check is falsey rather than undefined (slippage ? Math.round(slippage * 100) : undefined), also when the buyer supplies an explicit 0 - in which case Jupiter's server-side default applies. A buyer requesting 0% slippage on Solana therefore gets Jupiter's default rather than 0 bps; the EVM path forwards an explicit 0"*
 04-payments-and-networks.md

Not in line: The specification documents the divergence as accepted behaviour, so the same 0% input yields different execution semantics on Solana versus EVM.

Vulnerability Details

A buyer who deliberately sets 0% slippage gets Jupiter's default tolerance on Solana and a true 0% on EVM. The two chains therefore interpret the same buyer input differently.

Impact

A buyer's stated 0% slippage is not applied on Solana, so a swap they intended to fail rather than execute at any adverse rate can instead execute at Jupiter's default



tolerance. The effect is bounded by the aggregator default and the buyer's own wallet preview, but the cross-chain inconsistency is surprising.

Recommendation

Handle an explicit `0` consistently across chains by distinguishing not-supplied from zero with a defined check, for example `slippage === undefined ? default : Math.round(slippage * 100)`, so a deliberate 0% is either honoured on both chains or rejected on both with a clear message.

Status

Resolved

Conclusion

After Hashlock's analysis, the Moonberg project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.